

AFL Assistant — Executive Report

Domain-locked AFL chat + prediction assistant | Capstone Week 3, Day 5

System hardening, evaluation, API/UI deployment, and monitoring plan

Product Goal

Deploy a production-ready, AFL-only chat assistant that answers factual questions, retrieves historical player and team statistics, and generates probabilistic match-winner and top-player predictions. The system is designed to be deployed behind an API with a demo-ready UI, as if it were going live on a client property such as Web3Geeks.

Architecture

A LangGraph state machine processes each query through a rule-based intent classifier (router_node) and routes it to one of four branches — **prediction** (using two trained scikit-learn models: a match-winner classifier and a top-player regressor), **retrieval** (structured pandas lookups over historical match and player data), **factual** (predefined AFL-rules answers), or **refusal** (for off-topic or out-of-scope requests).

Prediction and retrieval outputs are passed through a **validation node**, which routes the result to a formatter, clarification prompt, or fallback message before producing the final response.

Day 5 wraps the graph in a **FastAPI /chat endpoint** with per-conversation session memory, structured logging, and basic rate limiting. An optional **Streamlit UI** is also provided for demonstrations. Every tool call is routed through resilience.py, which applies per-tool timeouts — 6 seconds for retrieval and 8 seconds for prediction — and returns a consistent error format.

Evaluation Results

A combined **31-case evaluation suite** (eval_suite.py) was executed across five categories using the project's afl_datasets/ and model_store/ directories. This represents the complete run rather than a partial evaluation.

Category	Pass	Fail	Pass Rate
Factual Q&A	5	0	100%
Scope guardrails (including 4 prompt-injection attempts)	10	0	100%
Retrieval	6	0	100%
Prediction sanity	6	0	100%
Multi-turn coherence	2	2	50%
Total	29	2	94%

Prompt-Injection Resistance

Six adversarial and abuse cases were tested: four prompt-injection attempts covering instruction override, persona override, fake system messages, and prompt-exfiltration attempts, along with empty-input and oversized-payload abuse guards.

All six cases were correctly refused, demonstrating that the system maintained its AFL-only scope and guardrails during testing.

Weakest Category: Multi-Turn Coherence (2/4)

Two distinct failure modes were identified:

- **M1 — "why do you think that" after a prediction:** The follow-up contains no entity, so the session-layer entity-carryover retry has nothing to reuse. The turn is classified as `off_topic` and refused, meaning the system loses the context of its own previous prediction.
- **M3 — "who won the most recent one" after a head-to-head:** The previous turn resolved a pair of teams (Richmond + Carlton), but the carryover mechanism currently stores and resends only a single entity. As a result, the team pair is lost and the follow-up is refused.

Two cases currently work correctly:

- **M2 — "what about his career average" after a player-stat lookup:** The session layer successfully carries the single player entity into the follow-up.
- **M4:** The system successfully recovers from an off-topic refusal and answers a subsequent valid AFL question instead of remaining stuck in refusal mode.

Highest-Value Improvement

The most direct improvement is to move entity/context carryover **inside the graph as a pre-router step**. This step would rewrite context-dependent follow-ups using entities resolved from the previous turn, including team pairs and, for M1, the previous intent and tool result, before the router performs classification.

This requires only one additional node rather than a complete redesign and directly targets the two failing multi-turn cases.

Naive-Baseline Comparison

`baseline_comparison.py` compares the trained match-winner model's accuracy and Brier score against a naive **"higher pre-match ladder points wins"** baseline using the same holdout split.

This is the **only metric in this report that has not yet been populated with real values**. It must be executed against the project's actual Day 1/2 holdout file and model artifacts before final accuracy and lift-over-baseline figures can be reported.

Known Limitations

- **Multi-turn memory is narrow, not general.** The current carryover mechanism handles a single missing entity in a follow-up but does not support entity-free follow-ups such as M1 or pair/compound entities such as M3. As a result, 2 of 4 multi-turn cases currently fail.
- **Data recency.** Predictions and statistics are only as current as the latest feature-snapshot refresh. The weekly snapshot refresh and retraining process are covered in the monitoring checklist.
- **Model accuracy ceiling.** Match outcomes contain inherent uncertainty caused by factors such as injuries, player selection, and in-game changes. The model's actual performance can only be quantified after `baseline_comparison.py` is run against the real holdout dataset.
- **Guardrail edge cases.** The router currently relies on regex and keyword-based rules rather than an LLM classifier. Novel phrasing outside the tested patterns may therefore be misrouted. `router_test.py` should be re-run whenever the pattern lists are modified.

Recommended Next Steps

1. **Implement general context-aware entity resolution in the graph** using a pre-router rewrite step so entity-free and pair-based follow-ups can be resolved correctly. This directly targets the two current multi-turn failures.

2. **Run baseline_comparison.py** against the real Day 1/2 holdout dataset and model artifacts to replace the provisional accuracy threshold with an actual model accuracy and measured lift over the naive baseline.
3. **Connect the structured logs** generated by api.py to a dashboard and alerting system based on the monitoring checklist before exposing the system to public traffic.
4. **Schedule the monthly retraining and weekly snapshot refresh** as automated calendar or scheduled jobs rather than relying on manual execution.